

Model battery plan — D5(b) and the rules-vs-model comparison

Not yet assigned to anyone. This lays out the two live-model experiments the team needs to run, the candidate models for each price tier (real, current OpenRouter pricing, checked 2026-09-18), and what has to happen before either can actually run. Bring this to the team to divide up.

Two separate experiments — do not conflate them

There are two different comparisons hiding under "run it live," and if a single run gets used for both, neither one means anything:

	Cross-model battery (D5b)	Rules-vs-model comparison
What varies	MODEL	DECISION_MODE
What's held fixed	DECISION_MODE = "model"	MODEL (one, cheap tier is fine)
Who runs it	6 team members, one model each	One person, one model, twice
Cost	~6 full batteries	~2 cheap-tier battery-equivalents
What it answers	Does model choice affect cost/accuracy?	Does letting code resolve the four gates beat letting the model decide?

Why DECISION_MODE must stay fixed at "model" for the cross-model battery: that mode is where model choice actually has something to affect — in "rules" mode the code decides the four gates regardless of which model is attached, so every model would converge to nearly identical results and the comparison would prove nothing.

Why the rules-vs-model comparison needs its own separate run: it's testing a completely different variable. Folding it into someone's battery slot would mean two things changed at once (model *and* mode), and the result couldn't be attributed to either one.

1 · The cross-model battery (D5b)

The brief's hard constraints

- **N–1 models for a team of N.** We're 7 → **6 members each run one full battery on a distinct model**; the 7th runs the v1-vs-v2 prompt pass instead.
- **At least 2 price tiers represented.**
- **No two members on the same model family.**

- **Everyone runs the identical eval set and identical v2 prompt** — model is the only thing that may differ.
- **Frontier tier, if used, runs on negative cases only** — a full frontier battery costs more than the brief's own quoted course allowance.

Candidate models, real OpenRouter pricing (checked 2026-09-18)

Family	Model (OpenRouter slug)	Input /M	Output /M	Tier
OpenAI	openai/gpt-5.6-luna	\$0.20	\$1.20	Cheap
DeepSeek	deepseek/deepseek-flash	\$0.15	\$0.60	Cheap
Mistral	mistralai/mistral-small-4	\$0.15	\$0.60	Cheap (open-weight)
Qwen	qwen/qwen3.8-flash	\$0.15	\$0.47	Cheap (open-weight)
Google	google/gemini-3.5-flash-lite	\$0.30	\$2.50	Cheap
Google	google/gemini-3.8-flash	\$0.75	\$3.75	Mid
Qwen	qwen/qwen3.8-max	\$2.00	\$6.00	Mid
Anthropic	anthropic/claude-sonnet-5	\$2.00	\$10.00	Mid
Anthropic	anthropic/claude-opus-5	\$5.00	\$25.00	Frontier
OpenAI	openai/gpt-6-astra	\$10.00	\$50.00	Frontier

Worth flagging for the D6 report: every one of these is more expensive than the brief's own cost-table reference price (\$0.10/\$0.40, checked 28 Aug 2026) — even the cheapest option here is ~50% pricier on both input and output. Real, current evidence that prices moved in the three weeks since the brief's table was set.

A suggested spread (still just a suggestion — pick as a team)

Slot	Family	Model	Tier
A	OpenAI	openai/gpt-5.6-luna	Cheap
B	DeepSeek	deepseek/deepseek-flash	Cheap
C	Mistral	mistralai/mistral-small-4	Cheap
D	Google	google/gemini-3.8-flash	Mid
E	Qwen	qwen/qwen3.8-max	Mid
F	Anthropic	anthropic/claude-opus-5	Frontier — negative cases only
G (7th member)	—	v1-vs-v2 prompt pass, on whichever model the team fixes	—

3 cheap + 2 mid + 1 frontier, 6 distinct families, 2+ tiers — satisfies every constraint above with room to argue about the exact picks.

2 · The rules-vs-model comparison

One person, one model (a cheap one — this doesn't need six keys), same 25/35/50-case eval set, run twice, on `BACKEND = "live"` with `--auto-approve` so `book` cases can actually complete (see item 3 below):

```
python3 run_eval.py --mode rules --all --auto-approve
python3 run_eval.py --mode model --all --auto-approve
```

Compare: pass rate, turns, tokens, cost, and — the most interesting number — how often each mode's conclusion disagrees with what `resolve_routing()` computed (logged automatically as `route_mismatch` / `route_mismatch_nonblocking` in the guardrail events). This is the actual evidence behind the team's D0/D6 architecture argument.

3 · What has to be true before anyone runs either experiment

1. **Your own** `OPENROUTER_API_KEY`, in your own shell — never a file, never committed.
2. **Add your model's real price to** `config.PRICES` in `A2_scaffold/config.py` before running — the code refuses to run with an unlisted model rather than silently mispricing it.
3. **Pass** `--auto-approve`. Since the autonomy-gate fix (see `docs/CHANGELOG.md`), a live run with no approval callback correctly *holds* every booking instead of auto-approving — every `book` case will show as "held" otherwise. `run_eval.py` now has a `--auto-approve` flag (added 2026-09-18, see `docs/CHANGELOG.md`) that simulates a human always saying yes, purely for measuring decision quality:

```
bash python3 run_eval.py --mode rules --all --auto-approve python3 run_eval.py
--mode model --all --auto-approve
```

It's a no-op on the scripted backend (already auto-approves internally) and only matters once `BACKEND = "live"`. It never changes `AUTONOMY` in `config.py` — the production gate is untouched. 4. **Revert** `BACKEND` **back to** `"scripted"` **locally before committing** — that has to stay the committed default for the whole repo. 5. **Save your results under your own filename** (e.g. `results_<model>.json`) rather than overwriting the shared `results.json` — other work (`cost_analysis.ipynb`) now reads that file as input.

Not decided yet — bring to the team

- Who takes which of the 6 model slots.
- Who takes the v1-vs-v2 prompt pass instead of a model slot.
- Who runs the rules-vs-model comparison (can be done by anyone, doesn't need to be one of the 6).

- A start date — `--auto-approve` (item 3 above) is now built, so this is no longer blocked on that; the only remaining prerequisites are each person's own API key and price entry (items 1–2).